

High-Performance Linpack Benchmark on Multiple Clusters: Optimization and Scaling Laws

LOC TUNG SU, University of New Mexico, USA

BENJAMIN WEBSTER, University of New Mexico, USA

GREGORY ZITER-GLASS, University of New Mexico, USA

With the ever-growing field of high performance computing, the demands for more and more computational power grow with every passing year. From the need to have a standardized measure of computation speed, the HPL linpack benchmark arose. At UNM, students were asked as a part of their HPC class to explore the benchmark on a few of the UNM-available clusters. This report dives into the discoveries found by running various differing-parameter runs on said clusters and observing the results.

ACM Reference Format:

Loc Tung Su, Benjamin Webster, and Gregory Ziter-Glass. 2026. High-Performance Linpack Benchmark on Multiple Clusters: Optimization and Scaling Laws. 1, 1 (June 2026), 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Introduction

At UNM there are four different computing clusters that are available to students that make a CARC account: Xena, Easley, Hopper, and a team cluster that students stand up for the fall 2025 Intro to High Performance Computing class. This report will detail discoveries made by our team when running the HPL linpack benchmark on two of the four available clusters. The benchmark is run with a .dat file that is read into the benchmark and then matrix operation speeds are recorded. This experiment explores how different parameters affect the Flop-rate of the system.

We ran 3 different experiments on the Easley and team clusters. We used the benchmark to explore the effects of strong scaling (Amdahl's law), weak scaling (Gustafson's law), and a different experiment of our choosing. Strong scaling refers to allocating more resources for a problem of fixed size, while weak scaling looks at performance if you increase the problem size over a fixed number of resources[1].

The parameters that we altered were the total problem size N , the block size NB , and the dimensions of the process grid P and Q . The strong scaling experiment was run on Easley and our team cluster Scoob, the weak scaling experiment was run on Easley, and the special experiment (NB scaling for future reference) was run on Easley and our team cluster.

Authors' Contact Information: Loc Tung Su, University of New Mexico, Albuquerque, NM, USA; Benjamin Webster, University of New Mexico, Albuquerque, NM, USA; Gregory Ziter-Glass, University of New Mexico, Albuquerque, NM, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM XXXX-XXXX/2026/6-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

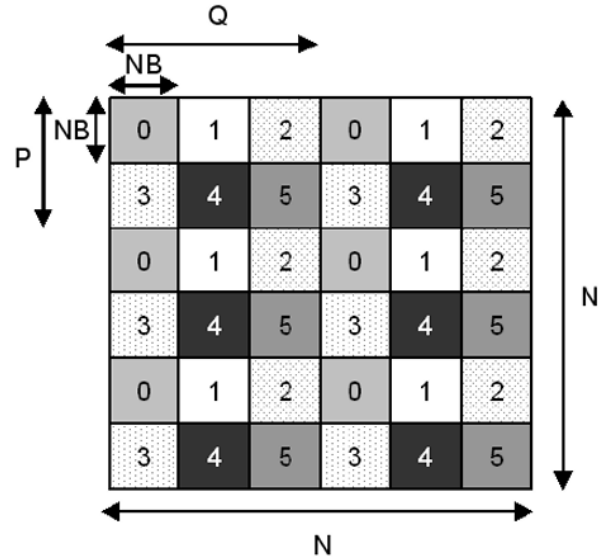


Fig. 1. Visual representation of the parameters that were altered in the experiment

Methods

0.1 General setup

With the provided High Performance Linpack (HPL) repository, in order to start collecting the data we're looking for, we first must modify the SLURM sbatch script, HPL_template.dat and the HPL_params.csv files. The HPL_params.csv file specifies the parameters for each job submission. Each submission can run multiple runs, and each row corresponds to a single run, with the columns representing: the block size (NB), the problem size (N), and the process grid dimensions P and Q . The HPL_template.dat is a file that is a place holder for the parameters we changed in HPL_params.csv and used to automatically generate a valid HPL.dat. With the provided HPL_template, we need to replace the N , NB , P , and Q with arbitrary indicators for our SLURM script, such as $\langle BLOCK_SIZE \rangle$, $\langle PROBLEM_SIZE \rangle$, $\langle P_SIZE \rangle$ and $\langle Q_SIZE \rangle$. Lastly, we need to modify the parameter_sweep_array.slurm to:

- Load in the correct version of gcc or intel and its respective HPL version. You can identify it through using the module spider hpl and module spider gcc
- Add a snippet of code that can read in the 2nd, 3rd and 4th column from the csv file.
- Provide code that substitutes the parameter value read from the PARAMS file into the HPL .data.

- OPTIONAL: Add more detail for the result output such as the problem size, P size, and Q size.

For the benchmarking process, in order to achieve great results, we need to figure out what is our best possible problem size. As instructed, we tried to have our problem size be 80% of the available memory. In example, given 2 nodes and 16GB on each node, the "max" problem size is:

$$\sqrt{\frac{1.6 \cdot 10^{10} B \cdot 2N \cdot 0.8}{8B}} \approx 58000$$

Surprisingly, that is not entirely the case. When we were trying to limit test on Easely, we discovered that the problem size can comfortably exceed the 80% threshold without running into any out-of-memory (OOM) issues.

0.2 Amdahl's strong scaling

For the strong scaling study, we designed an experiment to approximate Amdahl's law on both the CARC Easely cluster, and Scoob. We ran the experiment using the same fixed problem size (N) and block size (NB) with 1-unit increment for the process grid (1x1, 2x2, ..., 6x6). This was not without its limitations, because on Easely we kept running into OOM issues and on Scoob, we could only run the benchmark on a single node due to SLURM not working. The OOM issues on Easely initially appeared when trying to default the problem size to 80% of the available memory, despite later trying to lower our problem size into the 60-70% range. However, we quickly realized that the process grid dimension also plays a huge part in this benchmark process. We noticed this because all of the runs that produced OOM errors were runs that had the process grid at most 4x4, but as soon as we raised it to 5x5 or larger, the errors stopped. We ended up with N = 40000, NB = 256 as our sweet spot where it doesn't run into any OOM issues and with moderate runtime. HPL follows the Block Cyclic Data Distribution. Given the 3 parameters, NB, P, and Q where NB is the block size, and both P and Q represent the process grid dimensions. Say our problem size is 12x12, and we are working with a 2x3 process grid, this means each block in the process grid will be 4x6 entries, or 24 total entries. [2]. This represents the ideal case where each process handles the same amount of memory. This only happens when N is an exact multiple of $P \times NB$ and $Q \times NB$. For our benchmark experiment, we are limited to one parameter **NB** that represents both the rows and columns (NBxNB) of the block size. Hence, it is extremely likely that the amount of work gets distributed unevenly which means it can exceed the per-process memory limit, leading to an OOM error.

0.3 Gustafson's weak scaling

For the weak scaling study, we focused on the Easely cluster and designed an experiment to approximate Gustafson's model: as we increase the number of processes, we also increase the global problem size so that the local work per process stays in roughly the same ballpark. Instead of fixing the matrix dimension (N) as in the Amdahl experiment, we treated N as a function of the process grid size $P \times Q$.

We reused the same parameter sweep infrastructure as in the strong-scaling experiments. For Gustafson's scaling, we fixed the

block size at $NB = 192$ and varied the process grid over the square configurations

$$P \times Q \in \{1 \times 1, 2 \times 2, 3 \times 3, \dots, 8 \times 8\}.$$

For each grid size, we chose a corresponding matrix size N that (i) increased with the total process count $P \times Q$, and (ii) avoided out-of-memory errors on the Easely "general" partition. In practice this led to a sequence of problem sizes such as:

$$N \in \{20000, 28000, 32000, 34000, 38000, 52000\},$$

paired with grids from 1×1 up to 8×8 . These values were determined empirically: we started from the "safe" N suggested by the 80% memory rule of thumb and then pushed N upward until we consistently hit OOM or queue failures, backing off to the largest stable configuration for each grid.

This methodology allowed us to scale the global matrix size proportionally with the available computational resources while maintaining stable, repeatable runs. It ensured that each increase in process count corresponded to a meaningful increase in total work, making the collected results representative of weak-scaling performance in the spirit of Gustafson's Law.

Results

0.4 Amdahl's strong scaling

Our results are shown in table 1 and 2. We experience a significant drop in runtime as the process count increases on both Easely and Scoob. The speed-up increases from 1 to ≈ 27 times as the number of processes grows from 1 to 36. The results prove that our parallelization is very optimized. However, we can also see clearly that toward the end of our experiment where we have high process counts, the return speed-up is not as significant as the prior experiment. This is due to the rapid increase in the communication cost given the large amount of processes involved in the algorithm itself.

As for the Flop-rate we experience a similar trend where our growth is near-linear at small process counts, but becomes less linear in the 25 to 36 process range. Flop-rate has positive correlation with the problem size over the entire experiment. In LU factorization (HPL's algorithm), the bigger the problem size N, the more computations are required which leads to a higher Flop-rate. However, as we incorporate more processes, each process get assigned with a smaller block of the matrix which means less computation-per-process, hence a smaller increase in Flop-rate.

Moving on to the **f** values, the fraction of the program that cannot be parallelized and must remain serial: In our experiments, the **f** values ranged between 0.001 to 0.026 (0.1% and 2.6%). We then proceed to perform a curve_fit using the least squares method to find the best fit **f** values, which gives us $f \approx 0.01$ (1%). The **f** value plays a huge part in our speed up across all 6 experiments.

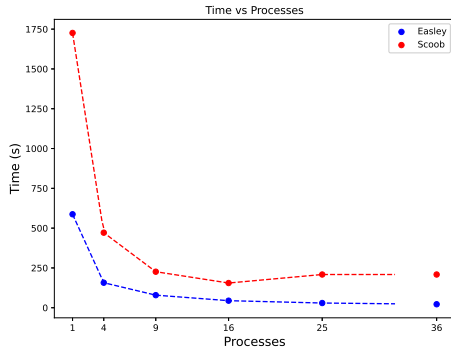
$$S(P) = \frac{1}{f + \frac{1-f}{P}}$$

$$\lim_{P \rightarrow \infty} S(P) = \frac{1}{f}$$

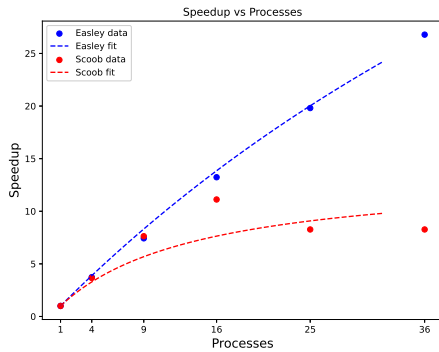
The smaller the serial fraction **f**, the larger the possible maximum speed up.

P	Q	GFlops	Time (seconds)	f	speedup
1	1	7.2667e+1	587.19	NaN	1
2	2	2.7149e+2	157.17	0.02355	3.736
3	3	5.3973e+2	79.06	0.02647	7.427
4	4	9.6177e+2	44.37	0.01393	13.235
5	5	1.4397e+3	29.64	0.01091	19.812
6	6	1.9469e+3	21.92	0.00983	26.785

Table 1. Performance results for varying thread counts with fixed NB (256) and fixed N (40000).



(a) Time differences as process counts increase



(b) Speed up differences as process counts increase

Fig. 2. Amdahl strong scaling

0.5 Gustafson's weak scaling

Our Gustafson experiment on Easley shows how HPL behaves when both the matrix size and number of processes increase together. The overall trend in Flop-rate is strongly increasing with process count. The single process 1×1 case achieved roughly 7×10^1 GLOP/s, while the largest 8×8 grid reached on the order of 2.8×10^3 GLOP/s. The intermediary grids filled this curve smoothly. This near monotonic growth is exactly what Gustafson's viewpoint emphasizes: as more processes are added, the system can sustain a significantly larger global workload.

Runtime behaves differently than in the Amdahl experiment. Because we intentionally increase N with the process grid size, the wall clock does not steadily shrink; rather, it stays within a relatively narrow band. The 1×1 run takes about 88 seconds, while most multi-process grids fall in the 20–60 second range, even though they are solving much larger linear systems. For example, moving from 1×1 at $N = 20000$ to 7×7 at $N = 38000$ more than doubles the matrix dimension, yet the runtime drops to around 19 seconds. The 8×8 case at $N = 52000$ takes somewhat longer (roughly 38 seconds), but this is still comparable to the single-process runtime despite involving several times more total floating-point work.

Using the 1×1 run as a baseline, we computed a “scaled speedup” $S(p) = P \cdot T_1/T_p$ that reflects Gustafson's perspective: what is the relative time to solution when we solve a larger problem on more processors? For mid-range grids (4×4 through 7×7) the scaled speedup grows substantially, indicating that we were able to leverage additional processes to tackle larger matrices without a proportional increase in runtime. At the largest scale (8×8), communication overhead and the more aggressive choice of N begin to show up as a deviation from ideal linear behavior, but the system still achieved multi-teraflop throughput and a substantial reduction in time-to-solution compared to the single-process run.

Overall, the weak scaling results align well with Gustafson's Law. When we increase both problem size and process count together, Easley maintains roughly similar runtimes while delivering higher total FLOP rates.

P	Q	GFLOPS	Time (s)	Scaled Speedup	Efficiency
1	1	6.9792e+01	88	1.000	1.000
2	2	2.6399e+02	62	2.839	0.710
3	3	5.6314e+02	44	6.000	0.222
4	4	9.3789e+02	31	11.355	0.177
5	5	1.2437e+03	24	18.333	0.147
6	6	1.5683e+03	26	20.308	0.094
7	7	2.0942e+03	19	32.421	0.094
8	8	2.7063e+03	38	18.526	0.036

Table 2. Gustafson weak-scaling results on Easley with varying process grids. Scaled speedup computed as $S_{\text{scaled}}(P) = \frac{P \cdot T_1}{T_P}$ with $T_1 = 88$ seconds.

0.6 NB scaling

Besides the Amdahl experiments (fixed problem size and fixed NB) and the Gustafson experiments (fixed NB with increasing N and $P \times Q$), we were also interested in conducting a third experiment that varies the block size NB). In this experiment, we collected performance data on Easley using NB values ranging from 32 to 4096, while keeping the problem size fixed at 40000. We ran each configuration across five different process grids, from 2×2 up to 6×6 .

Figure 3a represents our results. Expectedly, the larger the process grid, the better the Flop-rate since it improves data and computational efficiency. However, a larger NB does not necessarily translate to better performance. Across all grid configurations, the performance increases from $\text{NB} = 32$ to $\text{NB} \approx 128$, after which it drops

drastically from 128 to 4096. In every configuration we tested, the best performance for NB was when $NB \approx 2^7$. It is safe to say that $NB \approx 2^7$ is the most optimal NB size.

0.7 P and Q relation

Additionally, to further understand how the HPL benchmark performance varies under different process grid configurations, we collected Flop-rate data for all 81 combinations of PxQ ranging from 1x1 to 9x9. The problem size ($N = 40000$) and block size ($NB = 256$) were held fixed across all runs in order to maintain the integrity of performance collected.

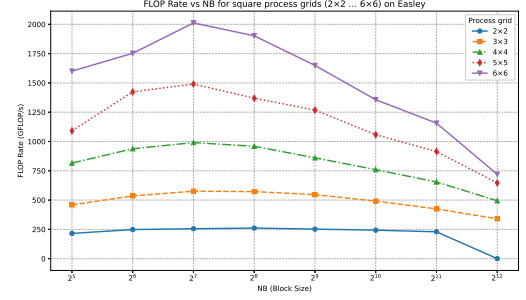
Figure 3a is a 9x9 heat map of the Flop-rate collected. Across the entire heat map, Flop-rate grows consistently as P and Q increase. The lowest performance happens at 1x1 with 73 GFLOP/s, and the best performance occurs at 9x9 with 3.5 TFLOP/s (a 48 times improvement). Additionally, we also noticed for both flatter and thinner process grids, the performance shows a noticeably lower performance in comparison to more well symmetrical process grid with the same number of processes. This can be seen in multiple places, i.e., 1x9, 9x1 giving worse performance than 3x3, and the same can be observed for 1x4, 4x1 against 2x2. This proves that the closer in value P and Q are, the better the performance we can achieve. This is especially true as this data was collected through the Easley cluster, and Easley uses Infiniband. The HPL algorithm under Infiniband prefers a more balance process grid whereas if it is under an Ethernet network, flatter process grid would be a more ideal choice.

0.8 Performance limit testing

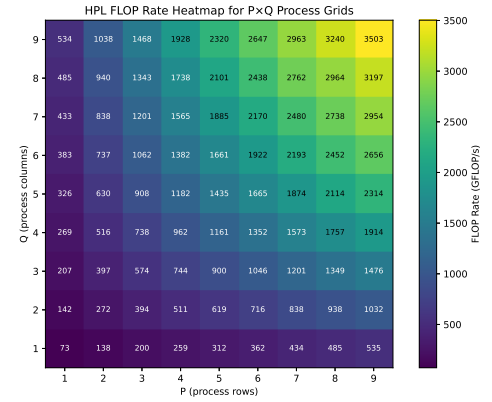
Using the information we gathered from our NB scaling and PxQ relation experiments, we ran some tests to see how far we can take N in order to increase the Flop-rate. These limit tests were performed on **general** partition, **10** nodes, **10** tasks per node, **50** cpus per node, with $NB = 128$ and $P \approx Q$, as this configuration provides the most optimal performance outcome as mentioned previously. Figure 3 displays some of our best runs. As we increase the problem size from 80000 to 130000, we experience improvement in the Flop-rate from 5.7 TFLOPS/s to 6.21 TFLOPS/s. Once again, we ran into OOM issues for $N = 140000$ and 150000, which indicates an implicit problem size limit given the provided configurations.

N	NB	P	Q	GFlops	Time (seconds)
80000	128	10	10	5.7160e+03	59.72
100000	128	10	10	5.9413e+03	112.21
120000	128	10	10	6.1177e+03	188.31
130000	128	10	10	6.2086e+03	235.91
140000	128	10	10	OOM	N/A
150000	128	10	10	OOM	N/A

Table 3. Performance results for varying thread counts with fixed NB (256) and fixed N (40000).



(a) NB scaling given fixed problem size from 2x2 to 6x6 size process grid.



(b) Relationship between P and Q

Fig. 3. Some interesting founding regard the NB parameter and the process grid PxQ.

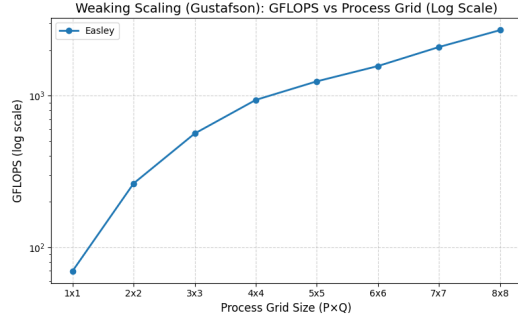
Conclusions

In this experiment, our team gathered a large amount of valuable information about how a small 2-node cluster performs the HPL benchmark on a single node, as well as how the same benchmark runs on a supercomputer meant for serious scientific and research computing. Throughout our runs, it was moreover proved that:

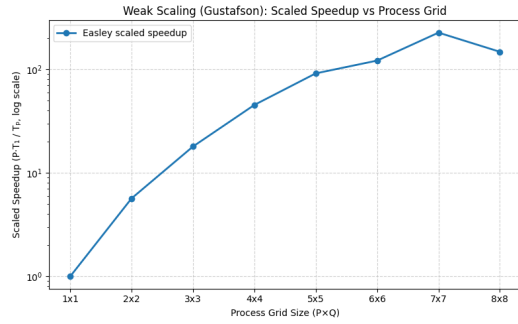
- By allocating more resources to a problem of fixed size, the maximum Flop-rate can be increased dramatically, leading to an immense drop in time taken to complete the benchmark.
- By allocating more resources, we can solve proportionally larger problems in roughly the same amount of time, leading to a large jump in the Flop-rate.

These conclusions reinforce that the parallel scaling laws, namely Gustafson's law of weak scaling and Amdahl's law of strong scaling hold true in modern computing clusters.

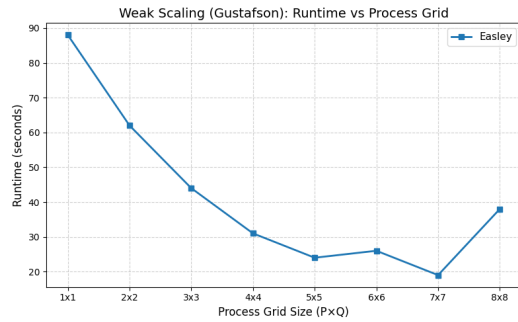
Due to the limited size of the Scoob cluster, which only contains two nodes, we were able to very plainly see a diminishing return when it came to running problems of larger sizes. Whereas on Easley, the cluster architecture had minimal effect on the performance of our jobs. This serves to highlight the importance of high-performance architecture in the age where problems get so astronomically large



(a) GFLOP/s increases nearly monotonically with larger process grids, demonstrating that Easley can sustain significantly higher total throughput as both N and process count grow.



(b) Scaled speedup follows Gustafson's prediction, rising sharply through mid-range grids before leveling off as communication overhead begins to dominate at the largest scale.



(c) Runtime remains within a relatively narrow band (20–60s) despite large increases in global matrix size, illustrating the central idea of weak scaling.

Fig. 4. Weak scaling (Gustafson) results on Easley for process grids from 1×1 to 8×8 .

that they take weeks if not months to run on even the most cutting-edge of modern computing clusters.

Author contributions

{Loc Su} works on the initial implementation of scaling and modifying the slurm script with Ben. Additionally, he worked on the

Amdahl's scaling from writing the slurm script to running the experiment on both Easley and team cluster, Scoob. He also worked on the **NB scaling, P & Q relation** and conduct the performance limit test. He also wrote the code to generate the plots for data visualization for all the experiments mentioned above. For the report, he wrote the methodology for the **general setup, Amdahl's strong scaling**, and the result for the **Amdahl's strong scaling, NB Scaling, P and Q relation**, and **Performance Limit Testing**.

{Benjamin Webster} Helped with initial implementation of scaling and initial slurm script, wrote script and parameters for Gustafson's experiments, conducted Gustafson's experiment. Wrote the code and generated graphs for Gustafson, wrote section dealing with Gustafson's weak scaling & results for Gustafson's weak scaling.

{Gregory Ziter-Glass} helped with writing and running the Amdahl's law and NB_Scaling experiments, along with consolidating the results into tables to be plotted. Also wrote the introduction, conclusion, and helped proofread the report.

References

- [1] Matthew Fricke. 2025. Lecture 19: Scaling. https://fricke.co.uk/Teaching/CS491_591_HPC_2025_Fall/Lectures/Lecture_19-Scaling.pdf. CS491/591: High Performance Computing, University of New Mexico, Fall 2025.
- [2] Susan Ostrouchov. 1995. Block Cyclic Data Distribution. <https://www.netlib.org/utk/papers/factor/node3.html>. Netlib.